JAVA ASSIGNMENT 1

NAME : PARTH KITCHLOO

PRN : 202401110014

BATCH A1

FUNDAMENTALS OF JAVA

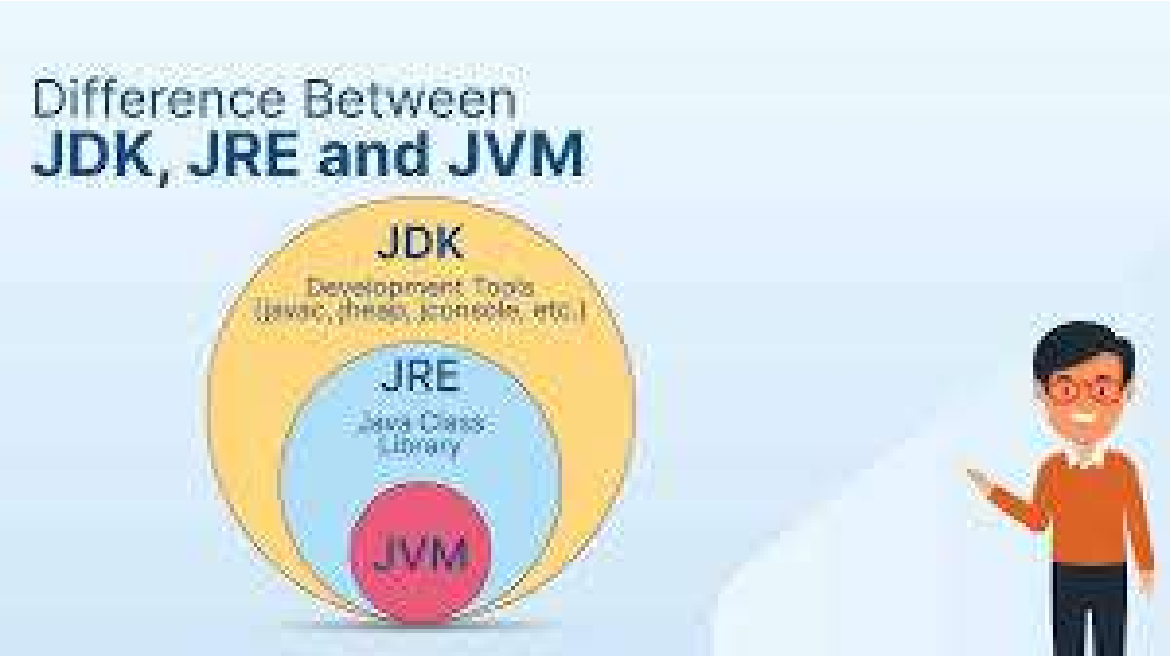
basics of Java

Java is a **programming language** and a **platform**. Java is a high-level, robust, object-oriented and secure programming language.

Java was developed by *Sun Microsystems* (which is now a subsidiary of Oracle) in the year 1995. *James Gosling* is known as the father of Java. Before Java, its name was *Oak*. Since Oak was already a registered company, so James Gosling and his team changed the name from Oak to Java.

Difference between JDK, JRE, and JVM

JDK, JRE, and JVM are three components form the core of the Java platform and are closely related, but each has a distinct role in Java development and execution. Before learning Java in depth, it is important to understand the difference between **JDK, JRE, and JVM**.

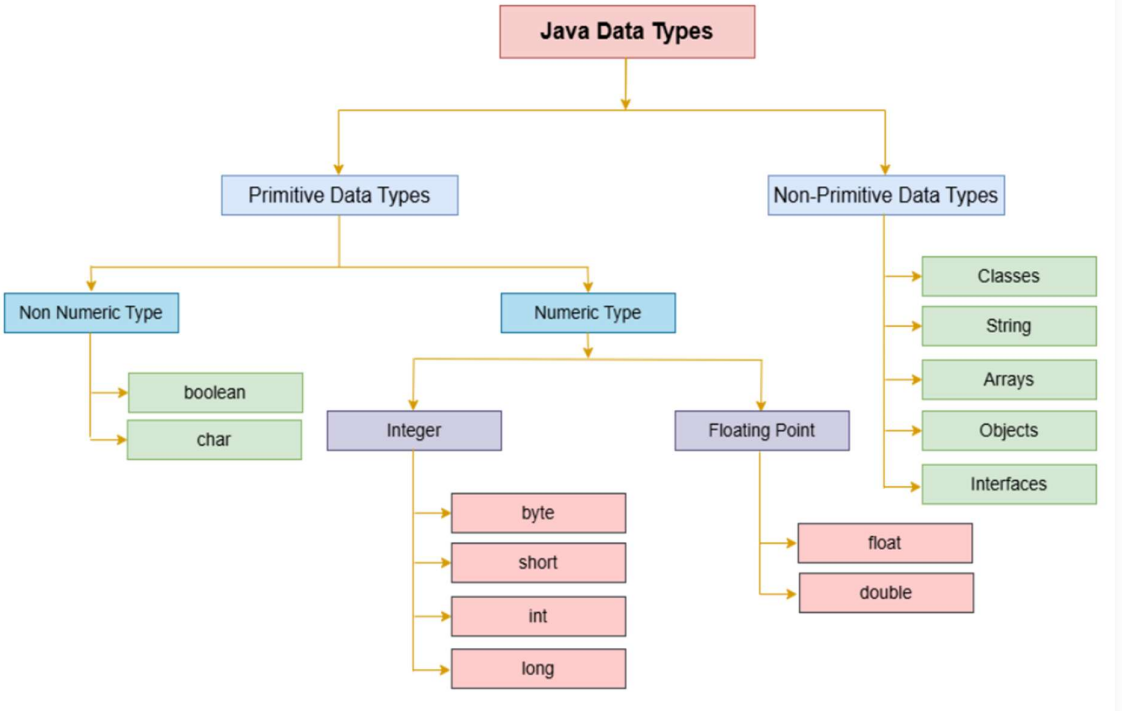


Difference Between C++ and Java

C++ is a powerful programming language that provides close control over system resources and memory management, whereas Java is designed to be simple, portable, and platform-independent. Let us understand the key differences between C++ and Java in different aspects such as memory handling, performance, platform dependency, and application areas.

Metrics	C	C++	Java
Programming Paradigm	Procedural language	Object-Oriented Programming (OOP)	Pure Object Oriented
Origin	Based on assembly language	Based on C language	Based on C and C++
Developer	Dennis Ritchie in 1972	Bjarne Stroustrup in 1979	James Gosling in 1991
Translator	Compiler only	Compiler only	Interpreted language (Compiler + interpreter)
Platform Dependency	Platform Dependent	Platform Dependent	Platform Independent
Code execution	Direct	Direct	Executed by JVM (Java Virtual Machine)
Approach	Top-down approach	Bottom-up approach	Bottom-up approach
File generation	.exe files	.exe files	.class files
Pre-processor directives	Support header files (#include, #define)	Supported (#header, #define)	Use Packages (import)
keywords	Support 32 keywords	Supports 63 keywords	50 defined keywords
Datatypes (union, structure)	Supported	Supported	Not supported
Inheritance	No inheritance	Supported	Supported except Multiple inheritance
Overloading	No overloading	Support Function overloading (Polymorphism)	Operator overloading is not supported
Pointers	Supported	Supported	Not supported
Allocation	Use malloc, calloc	Use new, delete	Garbage collector
Exception Handling	Not supported	Supported	Supported
Templates	Not supported	Supported	Not supported
Destructors	No constructor neither destructor	Supported	Not supported
Multithreading/ Interfaces	Not supported	Not supported	Supported
Database connectivity	Not supported	Not supported	Supported
Storage Classes	Supported (auto, extern)	Supported (auto, extern)	Not supported

Data Types in Java



Application

According to Sun Microsystems, 3 billion devices run Java. There are various devices where Java is currently used. Some of them are as follows:

1. Desktop Applications such as Acrobat Reader, media player, antivirus, etc.
2. Web Applications such as irctc.co.in, tpointtech.com, etc.
3. Enterprise Applications such as banking applications.
4. Mobile
5. Embedded System
6. Smart Card
7. Robotics
8. Games, etc.

Types of Java Applications

There are the following 4-types of applications that can be created using Java programming:

1) Standalone Application

Standalone applications are also known as desktop applications or window-based applications. These are traditional software that we need to install on every machine. Examples of standalone applications are Media players, antivirus, etc. AWT and Swing are used in Java for creating standalone applications.

2) Web Application

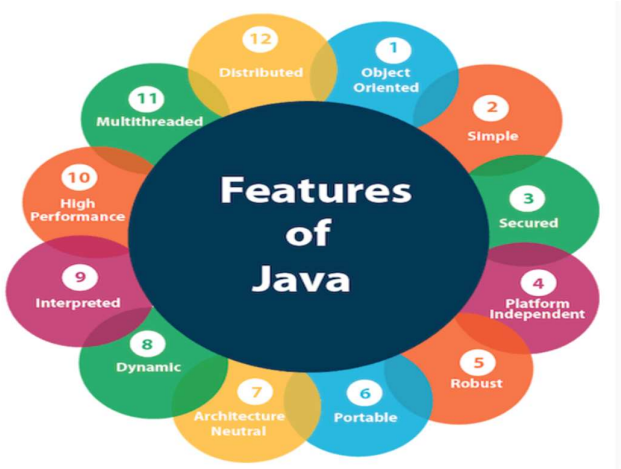
An application that runs on the server side and creates a dynamic page is called a web application. Currently, Servlet, JSP, Struts, Spring, Hibernate, JSF, etc. technologies are used for creating web applications in Java.

3) Enterprise Application

An application that is distributed in nature, such as banking applications, etc. is called an enterprise application. It has advantages like high-level security, load balancing, and clustering. In Java, EJB is used for creating enterprise applications.

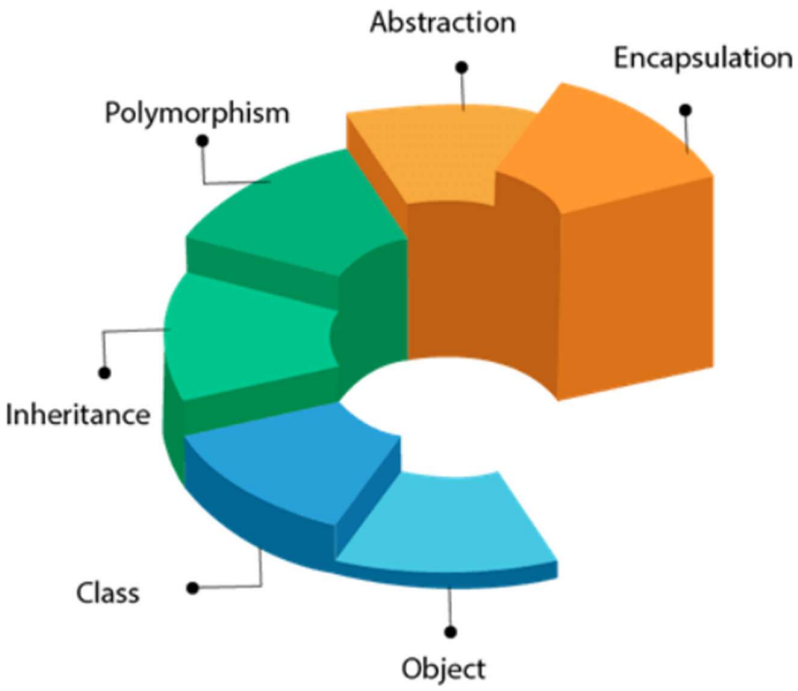
4) Mobile Application

An application that is created for mobile devices is called a mobile application. Currently, Android and Java ME are used for creating mobile applications.



Object-Oriented

Java is an object-oriented programming language where everything is an object. Object-oriented programming means that we organize our software as a combination of various types of objects that incorporate both data and behavior.



Java Classes and Objects

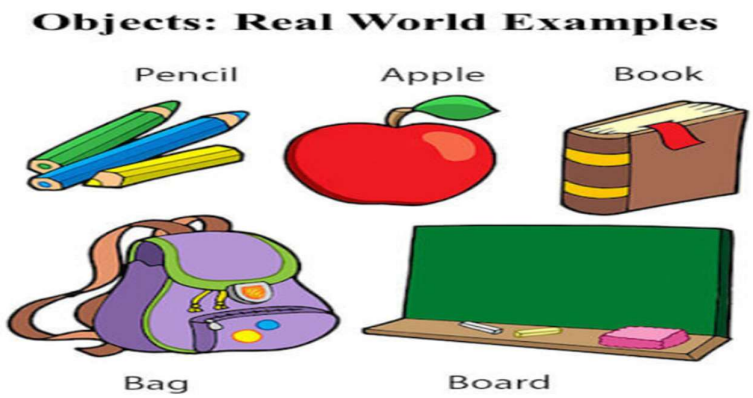
In object-oriented programming, classes and objects play a vital role in programming. These are the two main pillars of OOP (Object-Oriented Programming). Without classes and objects, we cannot create a program in Java. So, in this section, we are going to **discuss about classes and objects in Java.**

Java Class Syntax

```
class <class_name>{
   field;
   method;
}
```

Syntax of an Object

```
<class_name> variable=new <class_name>();
```




```java
class Dog {
  String name;
  void bark() {
    System.out.println(name + " says Woof!");
  }
}
public class Main {
  public static void main(String[] args) {
    Dog myDog = new Dog();    // Creating an object
    myDog.name = "Rocky";     // Assigning value to the object's field
    myDog.bark();             // Calling method on the object
  }
}
```

Demojava.java > ...

```java


import java.util.Scanner;

public class Demojava {

    Run | Debug
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Taking name input
        System.out.print(s: "Enter your name: ");
        String name = sc.nextLine();

        System.out.println("Name: " + name);

        // Taking number input
        System.out.print(s: "Enter the number: ");
        int no = sc.nextInt();

        System.out.println("Number: " + no);

        // String operations
        System.out.println("Uppercase: " + name.toUpperCase());
        System.out.println("Lowercase: " + name.toLowerCase());
        System.out.println("Length: " + name.length());


        sc.close();
    }
}


```